Arena Booking App

Phase 1

Team 3

Member Name	Member Roll #	Primary Responsibility
Muhammad Baqir	24L-0601	Use case descriptions 1-7
Ayyan Khan	24L-0912	Use case description 16-23
Sohaib Irshad	24L-0679	Analysis Class Diagram
Saadi Ali	24L-0704	Use case description 8-15
Hasan Shaigan	24L-0621	Use case diagram

Table of Contents

Table of Contents 2

1. Project Proposal 6

2. Use case Diagram
Error! Bookmark not defined.6

3.Use case Description 1 7

4. Use case Description 2 8

5.Use case Description 3 9

6.Use case Description 4 10

7.Use case Description 5 11

8.Use case Description 6 12

9.Use case Description 7 13

10.Use case Description 8 14

11.Use case Description 9 15

12.Use case Description 10 16

13.Use case Description 11 17

14.Use case Description 12 18

15.Use case Description 13 19

16.Use case Description 14 20

17.Use case Description 15 21

18.Use case Description 16 22

19.Use case Description 17 23

20.Use case Description 18 24

21.Use case Description 19 25

22.Use case Description 20 26

23.Use case Description 21 27

24.Use case Description 22 28

25.Use case Description 23 29

26.Class Analysis Diagram 30

BookMyCourt – Online Sports Arena Booking System

Project Proposal

Course Instructor:
Sir Zeeshan Nazar

Group Leader: Baqir
Zaidi – 24L0601

Group Members:
Hasan Shaigan – 24L0621

Ayyan Khan – 24L0912

Sohaib Irshad – 24L0679

Saadi Bhatti – 24L0704

National University of Computer and Emerging Sciences
Department of Computer Science

Lahore, Pakistan

---

1. Abstract

BookMyCourt is a web-based application designed to simplify the booking process of sports arenas for various sports such as cricket, football, padel, badminton, and futsal. The system allows users to browse available arenas, view time slots, and make bookings through a simple and intuitive interface. Arena owners can manage arena details, update availability, and view booking records. The goal of BookMyCourt is to replace manual booking methods with a centralized and easy-to-use online solution using basic web technologies.

---

2. Introduction

Booking sports arenas is usually done through phone calls, walk-ins, or messaging, which often leads to scheduling conflicts and lack of transparency. Players face difficulty finding available grounds, while arena owners struggle to manage bookings efficiently.

BookMyCourt provides a single online platform where users can easily search for sports arenas, check availability, and reserve time slots. The system focuses on simplicity and clarity to ensure that both users and arena owners can manage bookings without complexity or confusion.

---

3. Goals and Objectives

The objectives of this project are:

• To develop a web-based sports arena booking application
• To allow users to browse arenas based on different sports
• To provide time-slot based booking functionality
• To enable arena owners to manage arena availability and details
• To store and manage booking records efficiently

All objectives are defined at the same level of abstraction and focus on core system functionality.

---

4. Scope of the Project

The scope of BookMyCourt includes the development of a web application that supports online sports arena booking and basic management features. The frontend will be implemented using HTML, CSS, and JavaScript with a simple framework such as React. The backend will be developed using Node.js and Express to handle requests and manage application logic.

A lightweight database such as MongoDB or JSON-based storage will be used to store user data, arena information, and bookings. The project will support user authentication, booking management, and availability tracking. Features such as online payments, mobile applications, and advanced analytics are outside the scope of this project.

---

5. Features

Frontend Features
• User registration and login
• View list of sports arenas
• Filter arenas by sport type
• View available time slots
• Book an arena

Backend Features
• User authentication
• Arena management
• Booking management
• Availability tracking

Additional Features
• Booking confirmation display
• Basic dashboard for arena owners

---

6. Initial Study and Work Done So Far

An initial study was conducted by reviewing existing sports arena booking websites and reservation systems. Common features such as time-slot selection, booking confirmation, and arena listings were analyzed. Tutorials related to web development using React, Node.js, and Express were studied to understand implementation approaches.

Basic research on database design for booking systems was also carried out to understand how reservations and availability can be managed effectively. This study helped in finalizing the system architecture and feature set for BookMyCourt.

---

7. References

[1] MDN Web Docs, Introduction to Web Development
[2] React Official Documentation
[3] Node.js Official Documentation
[4] MongoDB Documentation

Group Members:

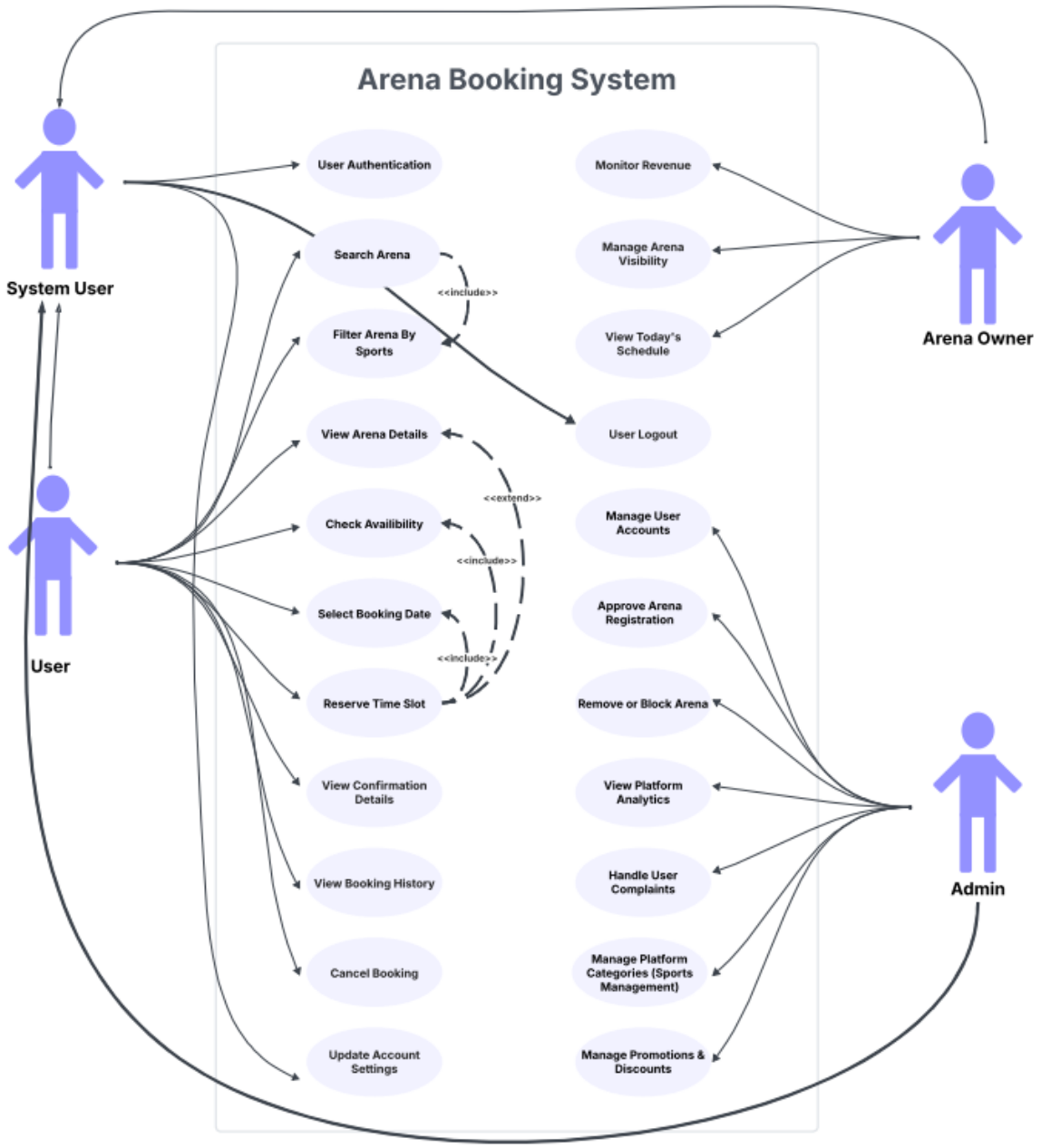
Hasan Shaigan – 24L0621

Ayyan Khan – 24L0912

Sohaib Irshad – 24L0679

Saadi Bhatti – 24L0704

Baqir Zaidi – 24L0601


Arena Booking System
System User
User
Arena Owner
Admin
User Authentication
Search Arena
Filter Arena By Sports
View Arena Details
Check Availibility
Select Booking Date
Reserve Time Slot
View Confirmation Details
View Booking History
Cancel Booking
Update Account Settings
<<include>>
<<extend>>
<<include>>
<<include>>
Monitor Revenue
Manage Arena Visibility
View Today's Schedule
User Logout
Manage User Accounts
Approve Arena Registration
Remove or Block Arena
View Platform Analytics
Handle User Complaints
Manage Platform Categories (Sports Management)
Manage Promotions & Discounts

Use Case Description 1:

Identifier	UC-01	
Name	User Authentication	
Summary	User enters credentials to access Player or Owner interface.	
Priority	High	
Actors	Player, Arena Owner	
Pre-condition(s)	User is on the Login screen.	
Post-condition(s)	User authenticated and redirected to dashboard.	
Typical Course of Action		
S#	**Actor Action**	**System Response**
1	User enters email and password.	System captures input.
2	User clicks Sign In.	System validates credentials.
		System loads dashboard.
Alternate Course of Action		
S#	**Actor Action**	**System Response**
1	User selects Owner Dashboard.	System opens owner interface.

Exceptions:
1. Invalid email or password.
2. User account suspended or blocked.

Use Case Description 2:

Identifier	UC-02
Name	Search Arena
Summary	Player searches for a specific venue using a text query.
Priority	High
Actors	Player
Pre-condition(s)	Player is logged in and on the Home screen.
Post-condition(s)	System displays filtered results matching the query.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Player clicks on the search bar.	System focuses the input field.
2	Player types an arena name (e.g., "Elite").	System updates the q state variable
3	-	System filters the ARENAS array and updates the UI list.

Exceptions:

1. No arenas match search query.
2. Search API timeout.

Use Case Description 3:

Identifier	UC-03
Name	Filter Arenas by Sport
Summary	Player filters the arena list by selecting a specific sport category.
Priority	Medium
Actors	Player
Pre-condition(s)	Player is on the Home screen.
Post-condition(s)	System displays arenas offering the selected sport.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Player selects a sport (e.g., "Padel").	System updates the sport state variable.
2	-	System updates the list to show arenas offering that sport.

Exceptions:
1. No arenas available for selected sport.
2. Sport category not found.

Use Case Description 4:

Identifier	UC-04
Name	View Arena Details
Summary	Player
Priority	High
Actors	Player
Pre-condition(s)	Player is on the Home screen.
Post-condition(s)	The Detail screen is displayed.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Player clicks on an Arena card.	System triggers the onArena function.
2	-	System sets screen to "detail" and renders the Detail component.

Exceptions:

1. Arena record not found (deleted or blocked).

Use Case Description 5:

Identifier	UC-05
Name	Check Availability
Summary	Player navigates to the calendar to see open booking slots.
Priority	High
Actors	Player
Pre-condition(s)	Player is on the Arena Detail screen.
Post-condition(s)	The Slots selection screen is displayed.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Player clicks "Check Availability".	System triggers the onBook function.
2	-	System navigates to the Slots selection component.

Exceptions:
1. No slots available for selected arena.

Use Case Description 6:

Identifier	UC-06
Name	Select Booking Date
Summary	Player selects a specific day for their reservation.
Priority	Medium
Actors	Player
Pre-condition(s)	Player is on the Slots screen.
Post-condition(s)	Time slots for the chosen day are displayed.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Player clicks on a date from calendar.	System updates the day state index.
2	-	System highlights the selected date and refreshes available slots.
		-

Exceptions:

1. No slots available for selected date.

Use Case Description 7:

Identifier	UC-07
Name	Reserve Time Slot
Summary	Player selects an available time and proceeds to confirm.
Priority	High
Actors	Player
Pre-condition(s)	Player is on the Slots screen with a date selected.
Post-condition(s)	Booking proceeds to confirmation.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Player clicks an "Available" time slot.	System updates the slot state.
2	Player clicks "Proceed to Book".	System triggers onConfirm and navigates to "confirmed" screen
		-

Alternate Course of Action		
S#	**Actor Action**	**System Response**
1	Player clicks a "Booked" slot.	System remains idle as the button is disabled.

Exceptions:

1. Session expired before confirmation.

Use Case Description 8:

Identifier	UC-08
Name	View Confirmation Receipt
Summary	Player views the digital ticket and booking ID.
Priority	Medium
Actors	Player
Pre-condition(s)	Player has successfully confirmed a booking.
Post-condition(s)	Confirmation UI is rendered.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Player reviews the confirmation card	System displays generated Booking ID and arena details.
2	Player clicks "Go to Home".	System resets view states and returns to the Home screen.

Exceptions:
1. Booking record not found.

Use Case Description 9:

Identifier	UC-09
Name	View Booking History
Summary	Player reviews all current and historical reservations.
Priority	Medium
Actors	Player
Pre-condition(s)	Player is logged in.
Post-condition(s)	List of bookings is displayed.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Player clicks "Bookings" in the bottom Nav bar.	System sets tab state to "bookings".
2	-	System renders Bookings component with history data array.

Exceptions:
1. No bookings found.

Use Case Description 10:

Identifier	UC-10
Name	Cancel Booking
Summary	Player cancels a confirmed reservation from their history.
Priority	High
Actors	Player
Pre-condition(s)	A "confirmed" booking exists in the history list.
Post-condition(s)	Booking status is updated to cancelled.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Player clicks "Cancel" link on a booking card	System updates status badge to "cancelled".

Exceptions:
1. Server error during status update.

Use Case Description 11:

Identifier	UC-11
Name	Monitor Revenue
Summary	Owner views total earnings for the current day.
Priority	High
Actors	Arena Owner
Pre-condition(s)	User is logged in as "owner".
Post-condition(s)	Revenue summary is displayed.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Owner opens the Dashboard	System displays the "Total Revenue" card with current value.

Exceptions:

1. Revenue data unavailable.

Use Case Description 12:

Identifier	UC-12
Name	Manage Arena Visibility
Summary	Owner toggles the arena availability for new bookings.
Priority	Medium
Actors	Arena Owner
Pre-condition(s)	Owner is on the Dashboard.
Post-condition(s)	Arena booking status is toggled.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Owner clicks the Visibility toggle switch.	System triggers setVis state change.
2	-	System updates status text to "Open" or "Closed".

Exceptions:

1. Database update fails.

Use Case Description 13:

Identifier	UC-13
Name	View Today's Schedule
Summary	Owner checks current day's bookings and client contact details.
Priority	High
Actors	Arena Owner
Pre-condition(s)	Owner is on the Dashboard.
Post-condition(s)	Today's schedule list is displayed.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Owner reviews "Today's Schedule" section.	System renders booked slots with client names and phone numbers.

Exceptions:

1. No bookings for today.

Use Case Description 14:

Identifier	UC-14
Name	Update Account Settings
Summary	Player modifies profile information or notification settings.
Priority	Low
Actors	Player
Pre-condition(s)	Player is on the Profile screen.
Post-condition(s)	Selected settings interface is opened.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Player clicks on "Notification Settings".	System navigates to the specific settings sub-menu.

Exceptions:

1. Duplicate email exists.

Use Case Description 15:

Identifier	UC-15
Name	User Logout
Summary	User signs out to end the current session.
Priority	Low
Actors	Player, Arena Owner
Pre-condition(s)	User is logged in.
Post-condition(s)	User is returned to the Login screen.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	User clicks "Sign Out" button. |	System triggers the logout() function.
2	-	System resets user state to null and clears navigation history

Exceptions:

1. Session already expired.

Use Case Description 16:

Identifier	UC-16
Name	Manage User Accounts
Summary	Admin views, suspends, activates or deletes Player or Owner accounts.
Priority	High
Actors	Admin
Pre-condition(s)	Admin is logged into admin panel.
Post-condition(s)	Target user account status is updated.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Admin opens “User Management”.	System displays list of accounts.
2	Admin selects a user.	-
3	Admin chooses action (Suspend, delete etc.).	System updates user account status.

Exceptions:

1. User not found.

Use Case Description 17:

Identifier	UC-17
Name	Approve Arena Registration
Summary	Admin reviews and approves newly registered arenas before they are publicized.
Priority	High
Actors	Admin
Pre-condition(s)	Owner has submitted a registration request.
Post-condition(s)	Arena is approved or rejected.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Admin opens “Pending Arena requests”.	System displays all requests.
2	Admins reviews documents and details.	-
3	Admin approves or rejects.	System updates arena status.

Exceptions:
1.Database update failure.

Use Case Description 18:

Identifier	UC-18
Name	Remove or Block Arena
Summary	Admin removes an arena due to policy violations or complaints.
Priority	High
Actors	Admin
Pre-condition(s)	Arena status is approved.
Post-condition(s)	Arena status becomes inactive.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Admin searches for arena.	System displays result.
2	Admin clicks profile of arena and presses “Block Arena”.	System disables booking access for the arena.

Exceptions:

1. Arena not found.

Use Case Description 19:

Identifier	UC-19
Name	View Platform Analytics
Summary	Admin monitors total bookings, total revenue, active users and growth trends.
Priority	High
Actors	Admin
Pre-condition(s)	Admin is logged in.
Post-condition(s)	Analytics dashboard displayed.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Admin clicks on dashboard page.	System displays all relevant information as in purpose.

Exceptions:

1. Analytics data unavailable.

Use Case Description 20:

Identifier	UC-20
Name	Handle User Complaints
Summary	Admin reviews and resolves complaints submitted by Player or Owners.
Priority	High
Actors	Admin
Pre-condition(s)	Complaint submitted in system.
Post-condition(s)	Complaint marked resolved.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Admin clicks on “Complaints”.	System displays complaints details arena-wise.
2	Admin reviews complaint details.	-
3	Admin takes action (warn owner/dismiss complaint).	System updates complaint status.

Exceptions:

1. Server error while updating complaint status.

Use Case Description 21:

Identifier	UC-21
Name	Manage Platform Categories (Sports Management)
Summary	Admin adds, edits, or removes sport categories (e.g., Cricket, Padel, Football).
Priority	Medium
Actors	Admin
Pre-condition(s)	Admin logged in.
Post-condition(s)	Updated sports list shown in UI filter.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Admin opens “Sport Categories”.	System displays current options.
2	Admin adds a sport.	System updates category list.

Exceptions:

1. Duplicate sport category.

Use Case Description 22:

Identifier	UC-22
Name	Manage promotions/discounts
Summary	Admin creates promotional campaigns or discount codes.
Priority	Medium
Actors	Admin
Pre-condition(s)	Admin logged in.
Post-condition(s)	Promotional codes becomes available for bookings.

Typical Course of Action		
S#	**Actor Action**	**System Response**
1	Admin opens “Promotions”.	System displays current options.
2	Admin creates a discount.	System activates promotion for selected arenas.

Exceptions:

1. Promotion already active.

Use Case Description 23:

Identifier	UC-23
Name	Forgot Password
Summary	User resets their account password if they forget their login credentials.
Priority	High
Actors	Player, Arena Owner, Admin
Pre-condition(s)	User is on the Login screen and has a registered account in the system.
Post-condition(s)	User can log in with the new password.

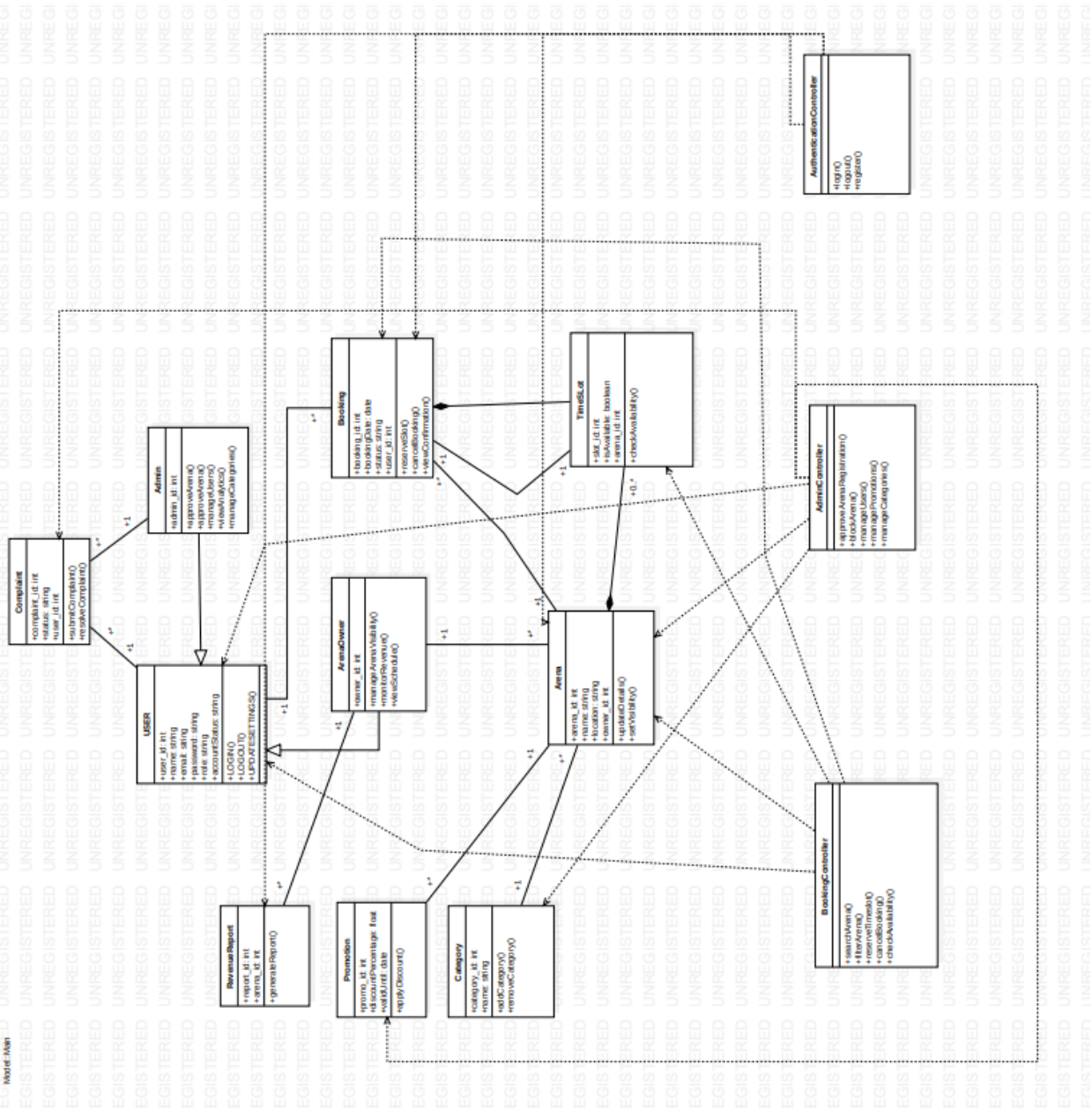
Typical Course of Action

S#	Actor Action	System Response
1	User clicks on "Forgot Password?" link.	System navigates to Password Reset screen.
2	User enters registered email address and presses “Submit”.	System validates email format and verifies from database.
3	-	System sends password reset link or OTP to registered email.
4	User clicks reset link / enters OTP.	System verifies OTP validity.
5	User enters new password and confirms it.	System validates password strength and updates password.

Alternate Course of Action

S#	Actor Action	System Response
1	User enters unregistered email.	System displays "Email not found" error message.

Exceptions:

1. Server unavailable.
2. Database connection failure.

Class Analysis Diagram